Guion de trabajo tutorizado

# MongoDB: Bases de datos documentales

## Gestión y visualización de datos masivos

*Profesor Dr. Domingo Savio Rodríguez Baena*
*Máster Big Data & Business Analytics (MBDA)*







Alumno: ÁLVARO RAMOS BERNABÉ

# ENUNCIADO DE LA ACTIVIDAD

*Guion oficial: «MBDA-Bases de datos orientadas a documentos-Actividad.pdf» (Campus Virtual UPO).*

## Objetivos

- Primera toma de contacto con el gestor de base de datos MONGODB
- Obtener conclusiones útiles del funcionamiento de MONGODB

## Ejercicios/Problemas

NOTA IMPORTANTE: Es crucial que todos los ejercicios estén bien explicados y que muestren el trabajo real que el/la estudiante ha llevado a cabo. Se pueden usar pantallazos para ello, pero que sean legibles y la resolución del ejercicio no se base únicamente en una sucesión de éstos.

**E1. (30 minutos - 0,8 puntos)**. Para trabajar con MongoDB es preciso realizar previamente su instalación. Es una instalación muy sencilla y que, además, no requiere gran cantidad de recursos.

La versión que instalaremos es la siguiente: MongoDB Enterprise Server 8.3.2 y la pueden encontrar en el siguiente enlace: https://www.mongodb.com/try/download/enterprise.

Podrá encontrar una guía de instalación aquí: https://docs.mongodb.com/manual/administration/install-enterprise/

Aunque los ejercicios que aparecen a continuación están descritos para ser ejecutados desde la línea de comandos, recomendamos instalar un cliente que pueda trabajar con la base de datos de forma más sencilla y visual. El mejor ejemplo es MongoDB Compass, que suele venir con la instalación de la base de datos.

Lleve a cabo la instalación de MongoDB, mostrando pantallazos del proceso y responda a esta pregunta:

- ¿Qué pasos hay que seguir para arrancar MongoDB? Recomendamos NO instalar la base de datos en modo SERVICIO.

**E2. (10 minutos - 0,3 puntos)**. Vamos a conectarnos a MongoDB a la vez que creamos una nueva base de datos llamada mbda. Para ello, y si usamos para trabajar la línea de comandos, hemos de ejecutar el siguiente comando:

```
$ use mbda
```

Si estamos en MongoDB Compass, hay que crear una nueva conexión, especificar manualmente sus características, indicar el nombre y seleccionar ‘localhost’ como servidor. Una vez creada la conexión, la establecemos, abrimos un MongoDBShell, ejecutamos use mbda y ya podemos trabajar.

A continuación, crearemos nuestra primera colección de documentos dentro de dicha base de datos. Se denominará towns.

El primer documento se refiere a la ciudad de Nueva York y tiene la siguiente estructura:

```
{
name: "New York",
population: 8491100,
last_census: ISODate("2014-12-31"),
famous_for: [ "statue of liberty", "food" ],
mayor : {
name : "Michael Bloomberg",
party : "I"
```

```
}
}
```

Como se puede observar, es la estructura de un objeto JSON, en la que las llaves {..} delimitan el comienzo y el final de dicho objeto. Cada objeto está formado por un conjunto de valores, cada uno de ellos definido a partir de una clave o atributo. Los corchetes [..] son utilizados para definir arrays.

Al no utilizar esquemas estáticos, MongoDB no necesita definir con comandos de tipo DDL (Data Definition Language) la estructura de cada colección antes de añadir datos. Simplemente, la colección se creará la primera vez que se añada un documento. El comando que se utiliza para crear un documento en una colección es el siguiente:

```
db.collection_name.insertOne( <document> )
```

Para comprobar que el documento ha sido insertado correctamente, solo hay que ejecutar el comando:

```
$ db.towns.find()
```

Este comando, utilizado para llevar a cabo consultas, mostrará todos los documentos de una colección concreta.

Inserte el documento referente a Nueva York en la colección towns y compruebe posteriormente que ha sido insertado correctamente. Muestre los comandos utilizados.

**E3. (10 minutos - 0,3 puntos)**. Se supone que una colección es un conjunto de documentos referidos al mismo concepto. Es decir, son el concepto análogo a la tabla pero con la importante salvedad de que no es necesario que los documentos sigan una estructura fija.

1- Intente insertar el siguiente documento en la colección towns, muestre el comenado utilizado y responda a las siguientes preguntas:

```
{
item: "ABC1",
details: {
model: "14Q3",
manufacturer: "XYZ Company"
},
stock: [ { size: "S", qty: 25 }, { size: "M", qty: 50
} ],
category: "clothing"
}
```

2- ¿Se podría almacenar en la colección towns un documento referente a cualquier otra cosa diferente a una ciudad y con una estructura completamente distinta?

3- ¿Qué implicaciones tiene este hecho?

**E4. (20 minutos - 0,6 puntos)**. Anteriormente, al ejecutar el comando db.towns.find( ) para mostrar los documentos de la colección towns, hemos podido comprobar que cada documento, además de contar con sus atributos, cuenta con un atributo _id de tipo ObjectID. Este atributo es la clave principal de cada colección y corresponde con un valor numérico secuencial.

Con respecto a este identificador único:

1- Explique las distintas partes que forman el atributo _id en una colección y su relación con el almacenamiento distribuido de MongoDB.

2- Indique la manera de poder modificar el _id de un documento.

**E5. (25 minutos - 0,7 puntos)**. Los comandos utilizados en MongoDB son llamadas a métodos implementados en JavaScript. Para llamar a esos métodos se utilizan distintos objetos. Existe el objeto

db, que hace referencia a la base de datos actual, o por ejemplo existe el objeto db.x, que hace referencia a la colección x de la base de datos db. El código de los métodos se puede consultar e incluso se pueden añadir métodos nuevos.

Añada a MongoDB una función en JavaScript denominada insertCity:

function insertCity(name, population, last_census, famous_for, mayor_info)

```
{
db.towns.insertOne({
name:name,
population:population,
last_census: ISODate(last_census),
famous_for:famous_for,
mayor : mayor_info
});
}
```

Esta función únicamente contiene una llamada al método db.towns.insertOne para añadir un nuevo documento a la colección towns.

Para hacerlo, únicamente pegue el código anterior en un terminal de mongodb o Compass y ejecútelo. A continuación, para llamar a la función, escriba lo siguiente:

insertCity(Atributo1, Atributo 2, …., Atributo N);

1- A continuación, utilice la función creada para añadir las siguientes ciudades:

```
name:”Punxsutawney”
population:6200
last_census: ISODate(“2008-01-31”)
famous_for: “phil the groundhog”
mayor: {name: “Jim Wehrle”}
name:”Portland”
population:582000
last_census: ISODate(“2007-09-20”)
famous_for: [“beer”, “food”]
mayor: {name: “Sam Adams”, party: “D”}
```

2- Muestre todos los comandos utilizados y explique brevemente los pasos seguidos.

3- Compruebe que funciona mostrando los documentos almacenados con el comando:

```
db.towns.find()
```

**E6. (15 minutos - 0,4 puntos)**. En MongoDB podemos establecer dos tipos de consultas: una simple y otra compuesta o ad hoc. Ambas se llevan a cabo a través del comando:

```
db.collection.find()
```

Para llevar a cabo una consulta simple sobre un documento, únicamente hay que conocer su identificador:

```
db.towns.find({"_id" : ObjectId("4d0ada1fbb30773266f39fe4") })
```

Este primer parámetro es un filtro que establece que buscamos un documento con un _id específico. Como ya conocemos, la propiedad _id es de tipo ObjectId y su contenido es de tipo cadena (tipo String).

Lo más lógico es que dicho identificador no sea conocido, por lo que también podemos filtrar por alguno de sus atributos:

```
db.towns.find({ population : 6200 })
```
El comando find() también puede aceptar un segundo parámetro, utilizado para llevar a cabo proyecciones, es decir, elegir que atributos del documento encontrado queremos mostrar por pantalla:

```
db.towns.find({ _id : ObjectId("4d0ada1fbb30773266f39fe4") }, {
name : 1 })
```
El valor 1 en el atributo name indica que queremos obtener el campo nombre del documento en cuestión. Un valor de 0, indica todo lo contrario.

1- Mediante una consulta simple, muestre por pantalla aquel documento cuyo atributo name es igual a 'Portland', incluyendo todos sus atributos menos el nombre.

2- Implemente la misma consulta, pero para que solo aparezca el atributo nombre.

3- En las dos consultas implementadas ¿Siempre aparece el campo _Id? En caso afirmativo, investigue si es posible obviarlo.

**E7. (15 minutos - 0,4 puntos)**. También es posible aplicar múltiples filtros al resto de atributos de los documentos. Estas son las llamadas consultas compuestas. Dichos filtros pueden basarse en operaciones de igualdad, de desigualdad, rangos o incluso mediante el uso de patrones. Por ejemplo:

```
db.towns.find(
{ name : /^P/, population : { $lt : 10000 } },
{ name : 1, population : 1 }
)
```
En esta consulta, existen dos condiciones de filtrado que se han de cumplir. Dichas condiciones de filtrado están unidas de forma lógica mediante un operador AND. En la primera, se ha utilizado una condición de filtrado de igualdad basada en una expresión regular compatible con Perl (/^P/). Dicha condición me permite buscar aquellos pueblos cuyos nombres empiezan por P. Con respecto a la segunda, se ha utilizado un operador condicional, los cuales tienen el formato:

```
field : {$op : value }
```
, donde $op es un operador de comparación. En nuestro ejemplo, se ha utilizado el operador $lt (lower than). Consulte los operadores de comparación, lógicos, etc. existentes en la URL: https://docs.mongodb.org/manual/reference/operator/query/.

1- Escriba una consulta que nos proporcione el nombre de aquellos documentos de la colección towns que se llamen 'Portland' o tengan una población de 6200 personas. Véase que, en este caso, en vez de conectar los filtros mediante un operador lógico AND, queremos aplicar un OR, es decir, obtendremos los documentos con tal que cumplan una de esas dos condiciones.

2- Escriba otra consulta que obtenga todos los nombres de todos los pueblos cuya fecha en el atributo last_census sea menor o igual al 31 de enero de 2008.

Nota: para especificar la fecha dentro del operador de comparación, utilice:

new Date("2008-01-31")}

**E8. (10 minutos - 0,3 puntos)**. Hemos comprobado cómo podemos establecer filtros a partir de valores concretos o de valores parciales (expresiones regulares).

Pero MongoDB permite utilizar rangos de valores:

```
db.towns.find(
{ famous_for : { $all : ['food', 'beer'] } },
{ _id : 0, name:1, famous_for:1 }
)
```

En este caso, estamos filtrando por un campo de tipo array y exigimos que en el array denominado famous_for se den todos los valores que hemos incluido en la lista. Ídem con el siguiente ejemplo pero en sentido contrario:

```
db.towns.find(
{ famous_for : { $nin : ['food', 'beer'] } },
{ _id : 0, name:1, famous_for:1 }
)
```

Pero la verdadera capacidad de MongoDB se encuentra en su habilidad de buscar en el interior de un documento hasta encontrar el campo de filtrado, independientemente de si ese campo se encuentra en un subdocumento embebido a gran profundidad (embedded documents). Para hacer referencia a dicho campo de filtrado, se utiliza una cadena con el siguiente formato:

‘nested_docname1.nested_docname2.nested_docname3...nested_docnameN. fieldName’

Es decir, ponemos delante el nombre de todos los documentos embebidos hasta llegar al nivel que nos interesa y, posteriormente, se añade el nombre del campo por el que queremos filtrar.

En la colección towns, los documentos que hacen referencia a ciudades tienen un subdocumento interno que habla sobre el alcalde de dicha ciudad.

> 1- Implemente una consulta que sea capaz de mostrar aquellos documentos de la colección towns en los que el alcalde tenga como valor ‘I’ en su campo party.

**E9. (20 minutos - 0,6 puntos)**. Una consecuencia de la estructura flexible dentro de los documentos es el hecho de que un documento puede o no tener un atributo por el cual estamos filtrando. Existe una manera de preguntar por este hecho y que justamente dicha pregunta constituya una forma más de filtrado.

Se lleva a cabo mediante el uso de comando $exists:

```
db.towns.find(
{ 'mayor.party' : { $exists : false } },
{ _id : 0, name : 1, mayor : 1 }
)
```

Esta consulta nos mostrará aquellos nombres de ciudades y los nombres de sus alcaldes que no tengan información sobre su partido almacenada, es decir, alcaldes que no tengan el atributo party. Un valor true utilizado con dicho comando nos indicaría justo lo contrario, es decir, que si tiene ese atributo.

A continuación, vamos a estudiar la directiva $elemMatch. 1- Para ello crearemos otra colección llamada countries e insertaremos tres documentos con los siguientes valores:

```
db.countries.insertOne({
_id : "us",
name : "United States",
exports : {
foods : [
{ name : "bacon", tasty : true },
{ name : "burgers" }
]
}
})
db.countries.insertOne({
_id : "ca",
name : "Canada",
exports : {
```

```
foods : [
{ name : "bacon", tasty : false },
{ name : "syrup", tasty : true }
]
}
})
db.countries.insertOne({
_id : "mx",
name : "Mexico",
exports : {
foods : [
{name : "salsa",tasty : true,condiment : true}
]
}
})
```

Como ha podido observar, en estos documentos se ha dado un valor concreto al campo _id, no dejando que MongoDB utilice su contador secuencial. Por lo que esta capacidad implica el peligro de poder insertar en la misma colección dos documentos con el mismo identificador.

2- Una vez creada la colección e insertado los documentos, describa la estructura interna de cada documento, indicando los distintos subdocumentos que lo componen ¿Cuántos niveles de enlazado interno tenemos?

3- Posteriormente, intente insertar otro documento de un país diferente, pero con un _id igual a alguno de los documentos ya insertados ¿Es posible?

**E10. (15 minutos - 0,4 puntos)**. A continuación, vamos a buscar un país que exporte bacon y además que tenga buen sabor (valor true del campo tasty):

```
db.countries.find(
{ 'exports.foods.name' : 'bacon', 'exports.foods.tasty' :
true },
{ _id : 0, name : 1 }
)
```

Si ejecutamos esta consulta, nos aparecen los países “United States” y “Canada”, pero ¿Es correcto este resultado? En el caso de “United States” podemos comprobar que contiene bacon y además es apetitoso (tasty=true). Pero en el caso de “Canada”, el valor del atributo taste es false para bacon. Entonces ¿Por qué aparece? Porque “Canada” contiene el valor true en el campo tasty, pero no para el bacon sino para otra comida: el sirope (syrup).

Esto se explica ya que la lista de atributos y valores de filtrado que aparece en el comando find() están afectados por el operador lógico AND pero a nivel del documento principal. Es decir, va a buscar algún documento principal de la colección countries que tenga un atributo exports.foods.name=’bacon’ y que tenga un atributo exports.foods.tasty=’true’, pero no tienen por qué pertenecer al mismo subdocumento (de nivel 2 en este caso). Para dar un mayor sentido a estas consultas se utiliza el comando $elemMatch:

```
db.countries.find(
{'exports.foods' : {
$elemMatch : {
name : 'bacon',
tasty : true }}
},
```

```
{ _id : 0, name : 1 }
)
```

De esta forma, no solo buscará documentos principales que contengan esos dos valores, sino que además se exige que pertenezcan al mismo subdocumento foods.

1- Implemente una consulta para obtener el nombre de aquellos países que tengan una comida sabrosa (tasty:true) y que además dicha comida tenga el atributo condiment ¿Qué país aparece?

**E11. (20 minutos - 0,6 puntos)**. Vamos a llevar a cabo una actualización de los documentos de la colección countries. Queremos añadir información sobre el estado al que pertenece cada ciudad o pueblo.

El comando updateOne consta de dos parámetros. Uno de ellos es un criterio de búsqueda a partir del cual localizar que documentos queremos que sean modificados.

1- Los dos siguientes comandos de modificación se distinguen por el efecto que producen en los documentos. Ejecútelos y describa dichas diferencias.

```
db.countries.updateOne(
{ name : "Mexico" },
{ $set : { "state" : "MX" } }
)
db.countries.replaceOne(
{ name : "Mexico" },
{ state : "MX" }
)
```

**E12. (20 minutos - 0,6 puntos)**. Además de $set, existen multitud de operadores que nos pueden ayudar a modificar un documento. Por ejemplo, el operador $inc.

1- Investigue como se utiliza el operador $inc y lleve a cabo una operación de modificación del documento de la colección towns cuyo atributo ‘name’ sea igual a ‘Punxsutawney’, de tal manera que incrementemos en 1000 su población actual y, además, el alcalde ahora se llamará ‘Jhon Doe’ y last_census tenga la fecha actual.

**E13. (20 minutos - 0,6 puntos)**. A continuación, estudiaremos las acciones de borrado. MongoDB es capaz de borrar aquellos documentos de una colección que se ajusten a un criterio de búsqueda concreto. Es importante saber que se borran documentos completos, no subdocumentos o atributos concretos. Además, se recomienda al usuario que se asegure de que los documentos que se van a borrar son en realidad los correctos. Para ello, simplemente hay que consultarlos primero.

Veamos un ejemplo: queremos borrar aquellos países que exportan bacon pero no sabroso (tasty:false). Lo primero que tendríamos que hacer sería consultar que documentos se borrarían:

```
db.countries.find(
{"exports.foods":{
$elemMatch:{"name":"bacon","tasty":false}

}
})
```

Una vez nos hemos cerciorado de que los documentos que aparecen son los que realmente queremos eliminar, ya podemos llevar a cabo el borrado.

1- Averigüe que comando ejecutar para borrar los países seleccionados anteriormente.

**E14. (50 minutos – 1,4 punto)**. MongoDB tiene la capacidad de optimizar sus consultas por medio de índices, capacidad que no es común a todos los sistemas gestores de base de datos NOSQL. No solo

proporciona índices implementados a partir de estructuras de recorrido eficiente del tipo B-Tree, sino que además ofrece índices que nos ayudan a organizar datos geoespaciales o hacer búsquedas sobre textos.

A continuación vamos a llevar a cabo un pequeño experimento para poder comprobar los efectos positivos que, a nivel de rendimiento, implica el uso de un índice. En primer lugar, hemos de añadir un número elevado de documentos a una colección. Para ello, utilizaremos este código:

```
function populatePhones(area,start,stop) {
for(var i=start; i < stop; i++) {
var country = 1 + ((Math.random() * 8) << 0); var num = (country * 1e10) + (area * 1e7) + i;
db.phones.insertOne({
_id: num,
components: {
country: country,
area: area,
prefix: (i * 1e-4) << 0,
number: i
},
display: "+" + country + " " + area + "-" + i
});
}
}
```

El cual crea una función capaz de crear números de teléfono y añadirlos a una nueva colección llamada phones. Vamos a crear 100000 números de teléfono de US, que van del 1-800-555-0000 al 1-800-565-9999. Para ello, llevaremos a cabo las siguientes llamadas:

```
populatePhones( 800, 5550000, 5650000);
```

Esta ejecución puede durar unos segundos/minutos. Después, para comprobar que los documentos se han creado y para obtener algunos de prueba, ejecutaremos una consulta sobre la colección phones, pero limitando el número de documentos a extraer a 2:

```
db.phones.find().limit(2)
```

Posible salida:

```
{ "_id" : 18005550000, "components" : { "country" : 1, "area" :
800, "prefix" : 555, "number" : 5550000 }, "display" : "+1 8005550000" }
{ "_id" : 88005550001, "components" : { "country" : 8, "area" :
800, "prefix" : 555, "number" : 5550001 }, "display" : "+8 8005550001" }
```

Siempre que una colección es creada, MongoDB crea un índice de forma automática sobre el atributo identificador '_id'. Comprueba la existencia del nuevo índice creado con el comando:

```
db.phones.getIndexes()
```

A continuación, vamos a ejecutar una consulta sobre uno de los dos documentos que hemos obtenido anteriormente. Pero esta vez, necesitamos recopilar estadísticas de la ejecución de la consulta. El comando explain() proporciona información sobre el plan de ejecución que MongoDB ha escogido para ejecutar una consulta. La opción executionStats nos proporcionará información tan interesante como el número de documentos accedidos o el tiempo en milisegundos empleado.

1- Ejecute la siguiente consulta y comente la información más importante relacionada con el plan de ejecución que aparecerá por pantalla: plan de ejecución utilizado (winningPlan), número de documentos consultados, tiempo en milisegundo, y cualquier otro dato que le resulte interesante.

Si comprueba que el tiempo en milisegundos es demasiado bajo (cercano a cero), utilice la función anterior para volver a aumentar el número de documentos de su colección. Así aumentará también el tiempo de ejecución de la consulta. Para ejecutar la consulta utilice el valor de _id que le ha aparecido en uno de los dos documentos que le apareció al comprobar que la creación de los teléfonos había sido correcta.

```
db.phones.find({display:"+8 800
5550000"}).explain("executionStats");
```

**E15. (40 minutos – 1 punto)**. La consulta que acabamos de ejecutar no utiliza el índice creado por MongoDB de forma automática sobre el atributo '_id', ya que se basaba en el valor de otro atributo. Vamos a crear un nuevo índice de forma expresa sobre dicho atributo.

1- Investigue que comando hemos de utilizar y construya un índice sobre el atributo display de tipo UNIQUE INDEX. Este índice forzará una restricción de valor único sobre ese atributo entre los documentos en la colección, evitando que a partir de este momento se pueda añadir un documento con un valor de display ya existente.

**E16. (40 minutos - 1 punto)**. Una vez creado el índice, vamos a ejecutar de nuevo la consulta anterior, extrayendo también las estadísticas.

1- Extraiga de nuevo la misma información que en el ejercicio anterior y compare la información que aparece en el plan de ejecución, dando una explicación justificada a las diferencias encontradas.

## Fecha y Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 8 de junio de 2026.

## Bibliografía

1. The MongoDB Manual (https://docs.mongodb.com/manual/)

# RESOLUCIÓN

Todas las órdenes se han ejecutado en **mongosh 2.12.0** contra **MongoDB Enterprise Server 8.3.11**. Las capturas reproducen la sesión interactiva: tras el *prompt* `mbda>` va el comando y debajo su resultado. No se ha usado MongoDB Compass porque el equipo de pruebas no tiene entorno gráfico; todo lo que se hace aquí puede hacerse igual desde el *MongoDB Shell* integrado en Compass.

## E1. Instalación de MongoDB

La guía pide la versión Enterprise 8.3.2. En la página de descargas la versión Enterprise vigente de esa rama es la **8.3.11**, una revisión posterior de la misma serie, que es la instalada. Se descarga el paquete `.tgz` para Ubuntu 24.04 x64 y la *MongoDB Shell* (`mongosh`), que desde la versión 6 se distribuye por separado.

```
aramber@localhost:~$ tar -xzf mongodb-linux-x86_64-enterprise-ubuntu2404-8.3.11.tgz && ls mongodb-linux-x86_64-enterprise-ubuntu2404-8.3.
11/bin
install_compass
mongocryptd
mongod
mongodecrypt
mongokerberos
mongoldap
mongos
aramber@localhost:~$ mongod --version | head -3
db version v8.3.11
Build Info: {
    "version": "8.3.11",
aramber@localhost:~$ mongosh --version
2.12.0
aramber@localhost:~$ mkdir -p /data/mongo/db /data/mongo/log

aramber@localhost:~$ mongod --dbpath /data/mongo/db --logpath /data/mongo/log/mongod.log --bind_ip 127.0.0.1 --port 27017 --fork 2>&1 | g
rep -v Shuffling
about to fork child process, waiting until server is ready for connections.
forked process: 17678
child process started successfully, parent exiting
aramber@localhost:~$ mongosh --quiet --eval 'db.runCommand({ping:1})'
{ ok: 1 }
```

**Figura 1.** Descompresión del paquete, versiones de `mongod` y `mongosh`, creación de directorios, arranque y comprobación con `ping`.

### ¿Qué pasos hay que seguir para arrancar MongoDB (sin instalarlo como servicio)?

- Descomprimir el paquete (o ejecutar el instalador MSI en Windows desmarcando la opción *Install MongoD as a Service*) y añadir la carpeta `bin` al `PATH`.
- Crear el directorio de datos y, si se quiere, el de logs. En este caso, `/data/mongo/db` y `/data/mongo/log`; en Windows, por ejemplo, `C:\data\db`.
- Lanzar el servidor indicando el directorio de datos: `mongod --dbpath /data/mongo/db` y, opcionalmente, `--logpath`, `--port 27017` y `--bind_ip 127.0.0.1`. En Linux `--fork` lo deja en segundo plano; en Windows el proceso queda abierto en su consola y no debe cerrarse.
- Conectarse con un cliente: `mongosh` (por defecto a `mongodb://localhost:27017`) o MongoDB Compass creando una conexión a `localhost`. `db.runCommand({ping:1})` devuelve `{ ok: 1 }`.
- Para pararlo: `db.getSiblingDB("admin").shutdownServer()` desde el shell (o Ctrl+C en la consola del `mongod`).

## E2. Base de datos mbda y colección towns

`use mbda` cambia a la base de datos `mbda`, que no se crea físicamente hasta que se inserta el primer documento. El `insertOne` crea implícitamente la colección `towns` y devuelve el `ObjectId` generado. `find()` muestra el documento, con la fecha convertida a `ISODate` y el subdocumento `mayor`.

```
use mbda
db.towns.insertOne({
  name: "New York", population: 8491100,
  last_census: ISODate("2014-12-31"),
  famous_for: [ "statue of liberty", "food" ],
  mayor : { name : "Michael Bloomberg", party : "I" }
})
db.towns.find()
```

```
test> use mbda
switched to db mbda
mbda> db.towns.insertOne({
...   name: "New York",
...   population: 8491100,
...   last_census: ISODate("2014-12-31"),
...   famous_for: [ "statue of liberty", "food" ],
...   mayor : { name : "Michael Bloomberg", party : "I" }
... })
{
  acknowledged: true,
  insertedId: ObjectId('6aae5f1a351dec69afe095c0')
}
mbda> db.towns.find()
[
  {
    _id: ObjectId('6aae5f1a351dec69afe095c0'),
    name: 'New York',
    population: 8491100,
    last_census: ISODate('2014-12-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

**Figura 2.** Inserción del documento de Nueva York y comprobación con `find()`.

## E3. Colecciones sin esquema fijo

**1)** Se inserta el documento del artículo `ABC1` en `towns` con `db.towns.insertOne({...})`. MongoDB lo acepta sin error y la colección pasa a tener 2 documentos con estructuras totalmente distintas.

```
test> use mbda
switched to db mbda
mbda> db.towns.insertOne({
...   item: "ABC1",
...   details: { model: "14Q3", manufacturer: "XYZ Company" },
...   stock: [ { size: "S", qty: 25 }, { size: "M", qty: 50 } ],
...   category: "clothing"
... })
{
  acknowledged: true,
  insertedId: ObjectId('6aae5f1ba3ae69f2a8fb9c12')
}
mbda> db.towns.find()
[
  {
    _id: ObjectId('6aae5f1a351dec69afe095c0'),
    name: 'New York',
    population: 8491100,
    last_census: ISODate('2014-12-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6aae5f1ba3ae69f2a8fb9c12'),
    item: 'ABC1',
    details: { model: '14Q3', manufacturer: 'XYZ Company' },
    stock: [ { size: 'S', qty: 25 }, { size: 'M', qty: 50 } ],
    category: 'clothing'
  }
]
mbda> db.towns.countDocuments()
2
```

**Figura 3.** Inserción del documento ABC1 en la colección towns.

**2) ¿Se podría almacenar en `towns` un documento sobre algo distinto a una ciudad y con otra estructura? Sí**, como se acaba de comprobar. Por defecto las colecciones de MongoDB no imponen esquema (*schema-less*): cada documento BSON puede tener campos, tipos y anidamientos distintos. El único campo obligatorio es `_id`.

**3) Implicaciones.** *Ventajas:* flexibilidad para evolucionar el modelo sin migraciones (`ALTER TABLE`), para almacenar datos heterogéneos o semiestructurados y para prototipar deprisa. *Inconvenientes:* la responsabilidad de la coherencia pasa a la aplicación. Las consultas deben contemplar documentos sin el campo (en E8 y E9 el documento ABC1 aparece como `{}` al proyectar campos que no tiene). Los índices y agregaciones pueden dar resultados inesperados si un mismo campo tiene tipos distintos, y se complica el mantenimiento y la calidad del dato. Por eso la buena práctica es que cada colección agrupe documentos del mismo concepto y, si hace falta, imponer reglas con **validación de esquema** (`$jsonSchema` en `db.createCollection` o `collMod`).

## E4. El identificador _id

**1) Partes del `_id` y relación con el almacenamiento distribuido.** El `ObjectId` es un valor binario de 12 bytes, que se muestra como 24 caracteres hexadecimales. En contra de lo que sugiere el enunciado, **no es un número secuencial global**. Lo genera normalmente el *driver* o el cliente, sin consultar al servidor, y se compone de:

| **Bytes** | **Contenido** | **Ejemplo (`6aae5f1a351dec69afe095c0`)** |
|---|---|---|
| 4 | Marca de tiempo en segundos desde la época Unix (momento de creación) | `6aae5f1a` → 19/09/2026 10:08:26 UTC |
| 5 | Valor aleatorio generado una vez por proceso (identifica máquina y proceso sin coordinación) | `351dec69af` |
| 3 | Contador incremental que empieza en un valor aleatorio, para | `e095c0` |

| | | |
|---|---|---|
| | distinguir los ObjectId generados en el mismo segundo | |

```
test> use mbda
switched to db mbda
mbda> const doc = db.towns.findOne({ name: "New York" })
mbda> doc._id
ObjectId('6aae5f1a351dec69afe095c0')
mbda> doc._id.getTimestamp()
ISODate('2026-09-19T10:08:26.000Z')
mbda> const hex = doc._id.toHexString()
mbda> ({ hex: hex, timestamp_4B: hex.substring(0, 8), aleatorio_5B: hex.substring(8, 18), contador_3B: hex.substring(1
8, 24) })
{
  hex: '6aae5f1a351dec69afe095c0',
  timestamp_4B: '6aae5f1a',
  aleatorio_5B: '351dec69af',
  contador_3B: 'e095c0'
}
mbda> new Date(parseInt(hex.substring(0, 8), 16) * 1000)
ISODate('2026-09-19T10:08:26.000Z')
mbda> db.towns.updateOne({ name: "New York" }, { $set: { _id: "nyc" } })
Uncaught MongoServerError: Plan executor error during update :: caused by :: Performing an update on the path '_id' wo
uld modify the immutable field '_id'
mbda> const copia = db.towns.findOne({ name: "New York" })
mbda> copia._id = "nyc"
nyc
mbda> db.towns.insertOne(copia)
{ acknowledged: true, insertedId: 'nyc' }
mbda> db.towns.deleteOne({ _id: doc._id })
{ acknowledged: true, deletedCount: 1 }
mbda> db.towns.find({ name: "New York" }, { name: 1 })
[ { _id: 'nyc', name: 'New York' } ]
```

**Figura 4.** Descomposición del ObjectId, extracción de la fecha con `getTimestamp()` e intento de modificar el `_id`.

**Relación con el almacenamiento distribuido:** como cada cliente, *driver* o `mongos` puede generar identificadores únicos de forma **independiente y sin coordinación central** (tiempo + aleatorio de proceso + contador), no hace falta una secuencia global que sería un cuello de botella en un clúster fragmentado (*sharding*) o replicado. Además, el prefijo temporal hace que los `_id` sean aproximadamente crecientes, lo que ordena los documentos por fecha de creación. Eso tiene una consecuencia de diseño: si se usa `_id` como **clave de fragmentación por rangos**, todas las inserciones nuevas van al mismo fragmento (*hot spot*). En ese caso se recomienda una clave de *shard* hash (`{_id: "hashed"}`) o una clave compuesta.

**2) ¿Cómo modificar el `_id`? No se puede actualizar**: es inmutable. Como muestra la captura, `updateOne(..., {$set: {_id: "nyc"}})` falla con *«Performing an update on the path '_id' would modify the immutable field '_id'»*. La única forma es **crear una copia del documento con el nuevo `_id` y borrar el original**: `findOne` → cambiar `_id` en la copia → `insertOne` → `deleteOne` del antiguo. Si hay más operaciones concurrentes, conviene hacerlo dentro de una transacción. En la captura, el documento de Nueva York pasa a tener `_id: "nyc"`.

## E5. Funciones JavaScript: insertCity

**Pasos:** (1) se define la función `insertCity` pegando su código en `mongosh`. Queda disponible en la sesión (`[Function: insertCity]`) y usa el objeto global `db`, que apunta a `mbda`. (2) Se llama dos veces con los datos de Punxsutawney y Portland. La función convierte la cadena de fecha con `ISODate()` y guarda `famous_for` tal cual: un *string* en el primer caso y un *array* en el segundo. (3) `db.towns.find()` confirma que las dos ciudades se han insertado con todos sus campos y un `ObjectId` generado.

```
test> use mbda
switched to db mbda
mbda> function insertCity(name, population, last_census, famous_for, mayor_info) {
...   db.towns.insertOne({
...     name: name,
...     population: population,
...     last_census: ISODate(last_census),
...     famous_for: famous_for,
...     mayor: mayor_info
...   });
... }
[Function: insertCity]
mbda> insertCity("Punxsutawney", 6200, "2008-01-31", "phil the groundhog", { name: "Jim Wehrle" })
mbda> insertCity("Portland", 582000, "2007-09-20", ["beer", "food"], { name: "Sam Adams", party: "D" })
mbda> db.towns.find()
[
  {
    _id: ObjectId('6aae5f1ba3ae69f2a8fb9c12'),
    item: 'ABC1',
    details: { model: '14Q3', manufacturer: 'XYZ Company' },
    stock: [ { size: 'S', qty: 25 }, { size: 'M', qty: 50 } ],
    category: 'clothing'
  },
  {
    _id: 'nyc',
    name: 'New York',
    population: 8491100,
    last_census: ISODate('2014-12-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6aae5f1f8e8a11cd3a39b77d'),
    name: 'Punxsutawney',
    population: 6200,
    last_census: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: 'phil the groundhog',
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6aae5f1f8e8a11cd3a39b77e'),
    name: 'Portland',
    population: 582000,
    last_census: ISODate('2007-09-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

**Figura 5.** Definición de `insertCity`, inserción de las dos ciudades y comprobación con `find()`.

La función sólo existe en la sesión actual del shell. Para reutilizarla se puede guardar en un fichero `.js` y cargarlo con `load()`, o en `~/.mongoshrc.js`. Las funciones almacenadas en el servidor (`system.js`) están obsoletas en MongoDB 8.

## E6. Consultas simples y proyecciones

```
test> use mbda
switched to db mbda
mbda> db.towns.find({ name: "Portland" }, { name: 0 })
[
  {
    _id: ObjectId('6aae5f1f8e8a11cd3a39b77e'),
    population: 582000,
    last_census: ISODate('2007-09-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
mbda> db.towns.find({ name: "Portland" }, { name: 1 })
[ { _id: ObjectId('6aae5f1f8e8a11cd3a39b77e'), name: 'Portland' } ]
mbda> db.towns.find({ name: "Portland" }, { _id: 0, name: 1 })
[ { name: 'Portland' } ]
```

**Figura 6.** Consultas sobre Portland con distintas proyecciones.

- **Todos los atributos menos el nombre:** `db.towns.find({ name: "Portland" }, { name: 0 })`. El 0 excluye el campo y se devuelven todos los demás.
- **Sólo el nombre:** `db.towns.find({ name: "Portland" }, { name: 1 })`. El 1 incluye el campo.

- **¿Siempre aparece `_id`?** Sí: en las dos consultas aparece aunque no se pida, porque `_id` se incluye por defecto en toda proyección. Se puede omitir excluyéndolo explícitamente: `db.towns.find({ name: "Portland" }, { _id: 0, name: 1 })` devuelve sólo `{ name: 'Portland' }`. `_id` es el único campo que admite combinar exclusión e inclusión en la misma proyección.

## E7. Consultas compuestas

```
test> use mbda
switched to db mbda
mbda> db.towns.find({ name: /^P/, population: { $lt: 10000 } }, { name: 1, population: 1 })
[
  {
    _id: ObjectId('6aae5f1f8e8a11cd3a39b77d'),
    name: 'Punxsutawney',
    population: 6200
  }
]
mbda> db.towns.find({ $or: [ { name: "Portland" }, { population: 6200 } ] }, { _id: 0, name: 1 })
[ { name: 'Punxsutawney' }, { name: 'Portland' } ]
mbda> db.towns.find({ last_census: { $lte: new Date("2008-01-31") } }, { _id: 0, name: 1, last_census: 1 })
[
  {
    name: 'Punxsutawney',
    last_census: ISODate('2008-01-31T00:00:00.000Z')
  },
  { name: 'Portland', last_census: ISODate('2007-09-20T00:00:00.000Z') }
]
```

**Figura 7.** Consulta del ejemplo (AND), consulta con `$or` y filtro por fecha con `$lte`.

- **Portland o 6.200 habitantes:** `db.towns.find({ $or: [ { name: "Portland" }, { population: 6200 } ] }, { _id: 0, name: 1 })` → Punxsutawney y Portland. `$or` recibe un array de condiciones y basta con que se cumpla una.
- **`last_census` ≤ 31/01/2008:** `db.towns.find({ last_census: { $lte: new Date("2008-01-31") } }, { _id: 0, name: 1, last_census: 1 })` → Punxsutawney (31/01/2008, incluida por el «igual») y Portland (20/09/2007). Nueva York (2014) queda fuera y el documento ABC1 no tiene el campo.

## E8. Arrays y subdocumentos

```
test> use mbda
switched to db mbda
mbda> db.towns.find({ famous_for: { $all: ['food', 'beer'] } }, { _id: 0, name: 1, famous_for: 1 })
[ { name: 'Portland', famous_for: [ 'beer', 'food' ] } ]
mbda> db.towns.find({ famous_for: { $nin: ['food', 'beer'] } }, { _id: 0, name: 1, famous_for: 1 })
[ {}, { name: 'Punxsutawney', famous_for: 'phil the groundhog' } ]
mbda> db.towns.find({ 'mayor.party': 'I' })
[
  {
    _id: 'nyc',
    name: 'New York',
    population: 8491100,
    last_census: ISODate('2014-12-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

**Figura 8.** `$all`, `$nin` y consulta por un campo de un subdocumento (`mayor.party`).

`$all` devuelve Portland (su array contiene *food* y *beer*). `$nin` devuelve los documentos cuyo `famous_for` no contiene ninguno de los dos valores: Punxsutawney y también **el documento ABC1, que no tiene el campo** (aparece como `{}` porque la proyección sólo pide `name` y `famous_for`). Es una consecuencia práctica del E3. Para el ejercicio pedido se usa la **notación de punto**: `db.towns.find({ 'mayor.party': 'I' })`, que devuelve Nueva York (Michael Bloomberg, independiente).

## E9. $exists, colección countries y _id duplicado

```
test> use mbda
switched to db mbda
mbda> db.towns.find({ 'mayor.party': { $exists: false } }, { _id: 0, name: 1, mayor: 1 })
[ {}, { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
mbda> db.countries.insertOne({ _id: "us", name: "United States", exports: { foods: [ { name: "bacon", tasty: true }, {
 name: "burgers" } ] } })
{ acknowledged: true, insertedId: 'us' }
mbda> db.countries.insertOne({ _id: "ca", name: "Canada", exports: { foods: [ { name: "bacon", tasty: false }, { name:
 "syrup", tasty: true } ] } })
{ acknowledged: true, insertedId: 'ca' }
mbda> db.countries.insertOne({ _id: "mx", name: "Mexico", exports: { foods: [ { name: "salsa", tasty: true, condiment:
 true } ] } })
{ acknowledged: true, insertedId: 'mx' }
mbda> db.countries.find()
[
  {
    _id: 'us',
    name: 'United States',
    exports: { foods: [ { name: 'bacon', tasty: true }, { name: 'burgers' } ] }
  },
  {
    _id: 'ca',
    name: 'Canada',
    exports: {
      foods: [ { name: 'bacon', tasty: false }, { name: 'syrup', tasty: true } ]
    }
  },
  {
    _id: 'mx',
    name: 'Mexico',
    exports: { foods: [ { name: 'salsa', tasty: true, condiment: true } ] }
  }
]
mbda> db.countries.insertOne({ _id: "us", name: "Uruguay", exports: { foods: [ { name: "asado", tasty: true } ] } })
Uncaught MongoServerError: E11000 duplicate key error collection: mbda.countries index: _id_ dup key: { _id: "us" }
```

**Figura 9.** `$exists`, inserción de los tres países e intento de insertar un `_id` repetido.

**1)** Se crean los tres documentos de `countries` con `_id` asignado a mano (`"us"`, `"ca"`, `"mx"`). La primera consulta de la captura (`$exists: false`) devuelve Punxsutawney, cuyo alcalde no tiene partido, y de nuevo el documento ABC1.

**2) Estructura interna.** Cada documento tiene **tres niveles** de anidamiento:

- **Nivel 1 (documento raíz):** `_id` (string), `name` (string) y `exports` (subdocumento).
- **Nivel 2:** el subdocumento `exports`, con un único campo `foods` de tipo *array*.
- **Nivel 3:** cada elemento del array `foods` es a su vez un subdocumento con `name` y, según el caso, `tasty` (booleano) y `condiment` (sólo en México). Estados Unidos tiene 2 comidas (bacon sabroso y burgers sin atributo `tasty`), Canadá 2 (bacon no sabroso y syrup sabroso) y México 1 (salsa sabrosa y condimento).

Es decir, hay **dos niveles de enlazado interno** por debajo del documento raíz (raíz → `exports` → elementos de `foods`) y la estructura de los elementos del nivel más profundo no es homogénea.

**3) ¿Se puede insertar otro país con un `_id` ya existente? No.** Toda colección tiene un índice único sobre `_id`, así que el intento de insertar «Uruguay» con `_id: "us"` falla con `E11000 duplicate key error … index: _id_ dup key: { _id: "us" }`. El «peligro» que menciona el enunciado no es que se dupliquen identificadores, sino que la aplicación asume la responsabilidad de generarlos únicos y recibirá errores si no lo hace. En cambio, `insertOne` con un `_id` automático nunca colisiona.

## E10. $elemMatch




**Figura 10.** Consulta sin y con `$elemMatch`, y consulta pedida (comida sabrosa con atributo condiment).

Se reproduce el problema: sin `$elemMatch` aparecen Estados Unidos y Canadá, porque las dos condiciones pueden cumplirse en elementos distintos del array. Con `$elemMatch`, sólo Estados Unidos. La consulta pedida exige que **el mismo elemento** de `foods` sea sabroso y tenga el atributo `condiment`:

```
db.countries.find(
  { 'exports.foods': { $elemMatch: { tasty: true, condiment: { $exists:
true } } } },
  { _id: 0, name: 1 }
)
```

**Aparece México**, cuyo único alimento, la salsa, es sabroso y condimento.

## E11. updateOne con $set frente a replaceOne

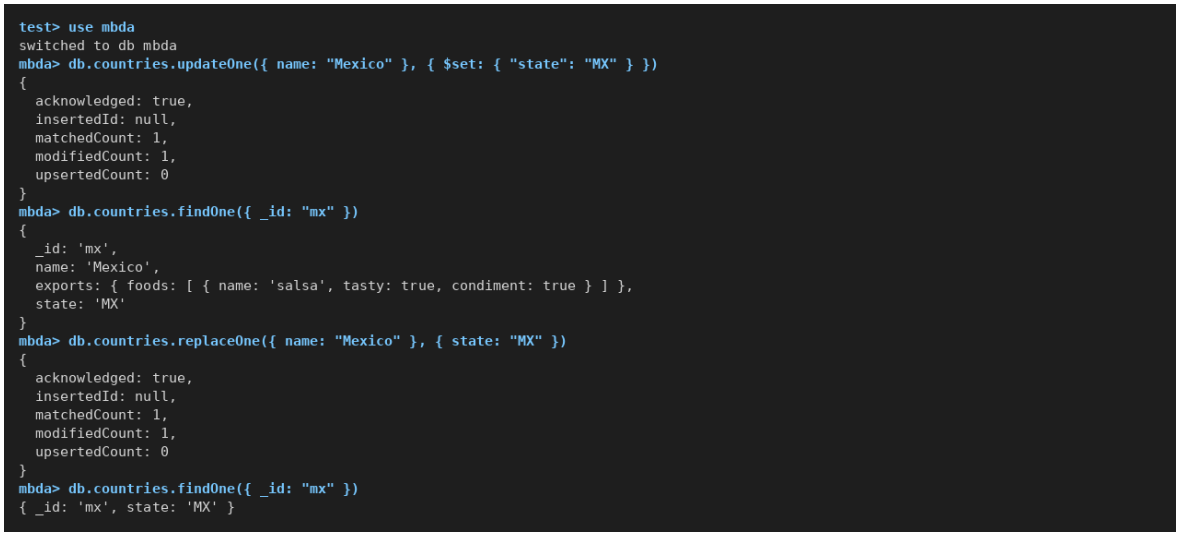



**Figura 11.** Efecto de `updateOne` con `$set` y de `replaceOne` sobre el documento de México.

- `updateOne({ name: "Mexico" }, { $set: { state: "MX" } })` **modifica** el documento: añade el campo `state` (o lo cambia si ya existe) y **conserva el resto** (`name`, `exports`).
- `replaceOne({ name: "Mexico" }, { state: "MX" })` **sustituye el documento completo** por el nuevo: sólo conserva el `_id`, que es inmutable. El resultado es `{ _id: "mx", state: "MX" }`: se han perdido `name` y `exports`.
- En ambos casos la respuesta indica `matchedCount: 1, modifiedCount: 1`, así que la diferencia sólo se ve al consultar el documento. En versiones antiguas, `update()` sin operadores `$` se comportaba como un reemplazo, y ese era un error frecuente. Desde

MongoDB 3.2, `updateOne` exige operadores de actualización y el reemplazo tiene su propio método, `replaceOne`.

## E12. Operador $inc

`$inc` suma (o resta, con valores negativos) una cantidad a un campo numérico de forma **atómica** en el servidor; si el campo no existe, lo crea con ese valor. Se combina en la misma operación con `$set` para el nombre del alcalde (notación de punto, sin borrar el resto del subdocumento) y con `$currentDate` para poner en `last_census` la fecha actual del servidor. Una alternativa equivalente sería `$set: { last_census: new Date() }`, con la fecha del cliente.

```
db.towns.updateOne(
  { name: "Punxsutawney" },
  { $inc: { population: 1000 },
    $set: { "mayor.name": "Jhon Doe" },
    $currentDate: { last_census: true } }
)
```

```
test> use mbda
switched to db mbda
mbda> db.towns.findOne({ name: "Punxsutawney" })
{
  _id: ObjectId('6aae5f1f8e8a11cd3a39b77d'),
  name: 'Punxsutawney',
  population: 6200,
  last_census: ISODate('2008-01-31T00:00:00.000Z'),
  famous_for: 'phil the groundhog',
  mayor: { name: 'Jim Wehrle' }
}
mbda> db.towns.updateOne({ name: "Punxsutawney" }, { $inc: { population: 1000 }, $set: { "mayor.name": "Jhon Doe" }, $
currentDate: { last_census: true } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
mbda> db.towns.findOne({ name: "Punxsutawney" })
{
  _id: ObjectId('6aae5f1f8e8a11cd3a39b77d'),
  name: 'Punxsutawney',
  population: 7200,
  last_census: ISODate('2026-09-19T10:08:43.083Z'),
  famous_for: 'phil the groundhog',
  mayor: { name: 'Jhon Doe' }
}
```

**Figura 12.** Documento antes (6.200 habitantes, Jim Wehrle, 2008) y después (7.200, Jhon Doe, fecha actual).

## E13. Borrado de documentos

Primero se consulta qué se va a borrar (sólo Canadá cumple la condición) y después se ejecuta el borrado con **el mismo filtro**:

```
db.countries.deleteMany({ "exports.foods": { $elemMatch: { "name": "bacon",
"tasty": false } } })
```

```
test> use mbda
switched to db mbda
mbda> db.countries.find({ "exports.foods": { $elemMatch: { "name": "bacon", "tasty": false } } })
[
  {
    _id: 'ca',
    name: 'Canada',
    exports: {
      foods: [ { name: 'bacon', tasty: false }, { name: 'syrup', tasty: true } ]
    }
  }
]
mbda> db.countries.deleteMany({ "exports.foods": { $elemMatch: { "name": "bacon", "tasty": false } } })
{ acknowledged: true, deletedCount: 1 }
mbda> db.countries.find({}, { _id: 1, name: 1 })
[ { _id: 'us', name: 'United States' }, { _id: 'mx' } ]
```

**Figura 13.** Consulta previa, borrado (`deletedCount: 1`) y comprobación de los documentos restantes.

Se usa `deleteMany` porque el criterio podría afectar a varios países; `deleteOne` borraría sólo el primero que encontrase. Se borran documentos completos: para eliminar un elemento del array habría que usar `updateOne` con `$pull`.

## E14. Rendimiento sin índice específico

Se define `populatePhones` y se generan **100.000 teléfonos** (1-800-555-0000 a 1-800-564-9999). `getIndexes()` confirma que sólo existe el índice automático `_id_`.

```
test> use mbda
switched to db mbda
mbda> function populatePhones(area, start, stop) {
...   for (var i = start; i < stop; i++) {
...     var country = 1 + ((Math.random() * 8) << 0);
...     var num = (country * 1e10) + (area * 1e7) + i;
...     db.phones.insertOne({
...       _id: num,
...       components: { country: country, area: area, prefix: (i * 1e-4) << 0, number: i },
...       display: "+" + country + " " + area + "-" + i
...     });
...   }
... }
[Function: populatePhones]
mbda> populatePhones(800, 5550000, 5650000)
mbda> db.phones.countDocuments()
100000
mbda> db.phones.find().limit(2)
[
  {
    _id: 58005550000,
    components: { country: 5, area: 800, prefix: 555, number: 5550000 },
    display: '+5 800-5550000'
  },
  {
    _id: 78005550001,
    components: { country: 7, area: 800, prefix: 555, number: 5550001 },
    display: '+7 800-5550001'
  }
]
mbda> db.phones.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

**Figura 14.** Carga de 100.000 documentos, muestra de 2 documentos e índices de la colección.

Se consulta uno de los documentos obtenidos (`display: "+5 800-5550000"`) con `explain("executionStats")`. El valor de `display` sigue el formato que genera la propia función: país, espacio, área, guion y número. Del plan se extrae lo más relevante:




**Figura 15.** Plan de ejecución y estadísticas de la consulta por `display` sin índice.

- **winningPlan: `COLLSCAN`**, un recorrido completo de la colección, aplicando el filtro `display $eq "+5 800-5550000"`. No hay índice sobre `display` y el de `_id` no sirve para este filtro.
- **totalDocsExamined: 100.000** para **nReturned: 1**: se leen todos los documentos para devolver uno (eficiencia 1/100.000).
- **totalKeysExamined: 0**: no se usa ningún índice.
- **executionTimeMillis: 35 ms.** Parece poco, pero crece de forma lineal con el tamaño de la colección. En la actividad In-Memory II se comprueba que llega a unos 120 ms con 400.000 documentos.

## E15. Índice UNIQUE sobre display

```
db.phones.createIndex({ display: 1 }, { unique: true })
```




**Figura 16.** Creación del índice único, listado de índices y comprobación de la restricción.

`createIndex` crea un índice B-tree ascendente sobre `display`. Con `unique: true` impide valores repetidos. La creación sólo es posible porque no había duplicados; si los hubiera, fallaría. Para comprobar la restricción, se intenta insertar un documento con un `display` existente y MongoDB lo rechaza con `E11000 duplicate key error … index: display_1`.

## E16. Rendimiento con índice

```
test> use mbda
switched to db mbda
mbda> var e2 = db.phones.find({ display: "+5 800-5550000" }).explain("executionStats")
mbda> e2.queryPlanner.winningPlan
{
  isCached: false,
  stage: 'EXPRESS_IXSCAN',
  keyPattern: '{ display: 1 }',
  indexName: 'display_1'
}
mbda> ({ nReturned: e2.executionStats.nReturned, executionTimeMillis: e2.executionStats.executionTimeMillis, totalKeys
Examined: e2.executionStats.totalKeysExamined, totalDocsExamined: e2.executionStats.totalDocsExamined, stage: e2.execu
tionStats.executionStages.stage, indexName: e2.executionStats.executionStages.indexName, keysExamined_stage: e2.execut
ionStats.executionStages.keysExamined })
{
  nReturned: 1,
  executionTimeMillis: 0,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  stage: 'EXPRESS_IXSCAN',
  indexName: 'display_1',
  keysExamined_stage: 1
}
```

**Figura 17.** Plan de ejecución de la misma consulta tras crear el índice.

| Métrica | Sin índice (E14) | Con índice único (E16) |
|---|---|---|
| Plan (winningPlan) | `COLLSCAN` | `EXPRESS_IXSCAN` sobre `display_1` |
| Claves de índice examinadas | 0 | 1 |
| Documentos examinados | 100.000 | 1 |
| Documentos devueltos | 1 | 1 |
| Tiempo (executionTimeMillis) | 35 ms | 0 ms (< 1 ms) |

**Explicación.** Con el índice, el optimizador ya no recorre la colección: desciende por el árbol B del índice `display_1` hasta la única entrada con ese valor (coste logarítmico, del orden de 17 comparaciones para 100.000 claves) y lee **un único documento**. Como el índice es **único** y la consulta es de igualdad exacta, MongoDB 8 usa además la ruta rápida **`EXPRESS_IXSCAN`**. Esta ruta se salta la planificación completa, porque sabe que habrá como mucho un resultado. Por eso el tiempo baja de 35 ms a menos de 1 ms y los documentos examinados pasan de 100.000 a 1. La contrapartida del índice es el espacio adicional y un pequeño coste extra en cada inserción o modificación de `display`, que ahora también debe actualizar el índice y comprobar la unicidad.

## Referencias

[1] The MongoDB Manual. https://www.mongodb.com/docs/manual/

[2] MongoDB Manual – ObjectId. https://www.mongodb.com/docs/manual/reference/method/ObjectId/

[3] MongoDB Manual – Explain Results (COLLSCAN, IXSCAN, EXPRESS_IXSCAN). https://www.mongodb.com/docs/manual/reference/explain-results/